

PyTorch Curriculum

Beginner → Advanced | Structured PyTorch learning roadmap

Goal: Build strong practical PyTorch skills from tensors and autograd through model development, training, optimization, and deployment.

LEVEL 1 — PYTORCH FUNDAMENTALS

1. *Introduction to PyTorch*

- What is PyTorch?
- PyTorch ecosystem
- PyTorch vs other frameworks
- Installation
- CPU vs GPU
- PyTorch workflow

2. *Tensors*

- Creating tensors
- Tensor attributes
- Shape
- Dimensions
- Data types
- Device
- Indexing
- Slicing
- Reshaping
- `view()`
- `reshape()`
- `squeeze()`
- `unsqueeze()`
- `permute()`
- `transpose()`

3. *Tensor Operations*

- Arithmetic operations
- Element-wise operations
- Matrix multiplication
- Dot product
- Broadcasting
- Concatenation
- Stacking
- Reduction operations
- `einsum` — basic understanding

4. *Tensor Devices*

- CPU tensors
- CUDA tensors
- Moving tensors between devices
- Device management
- GPU availability
- CPU/GPU synchronization

LEVEL 2 — AUTOGRAD

5. *Automatic Differentiation*

- What is autograd?
- Computational graphs
- `requires_grad`
- Gradients
- `backward()`
- `.grad`
- Gradient accumulation
- `detach()`
- `torch.no_grad()`

6. *Computational Graphs*

- Dynamic computation graphs
- Forward computation
- Backward computation
- Gradient flow
- Leaf tensors
- Non-leaf tensors

7. *Gradient Management*

- Zeroing gradients
- Gradient accumulation
- Gradient clipping
- Detaching tensors
- Gradient tracking vs inference

LEVEL 3 — NEURAL NETWORK API

8. *torch.nn*

- `nn.Module`
- `nn.Parameter`
- `nn.Sequential`
- `nn.ModuleList`
- `nn.ModuleDict`

9. Layers

- nn.Linear
- nn.Conv1d
- nn.Conv2d
- nn.Conv3d
- nn.BatchNorm
- nn.LayerNorm
- nn.Dropout
- Pooling layers
- Embedding layers

10. Activation Functions

- ReLU
- Sigmoid
- Tanh
- Softmax
- GELU
- LeakyReLU

11. Building Custom Models

- Defining a class
- `__init__()`
- `forward()`
- Custom layers
- Nested modules
- Model architecture

LEVEL 4 — DATASET & DATALOADER

12. Dataset

- `torch.utils.data.Dataset`
- `__len__()`
- `__getitem__()`
- Custom datasets

13. DataLoader

- Batching
- Shuffling
- Workers
- `num_workers`
- `pin_memory`
- `drop_last`
- Batch size

14. Data Pipelines

- Loading data
- Preprocessing
- Transformations
- Custom collate functions
- Efficient data loading

LEVEL 5 — TRAINING MODELS

15. Loss Functions

- nn.MSELoss
- nn.L1Loss
- nn.CrossEntropyLoss
- nn.BCELoss
- nn.BCEWithLogitsLoss
- Custom loss functions

16. Optimizers

- SGD
- Momentum
- Adam
- AdamW
- RMSProp
- Optimizer parameters

17. Training Loop

- Forward pass
- Loss calculation
- Backward pass
- Optimizer step
- Gradient reset
- Epochs
- Batches

18. Validation Loop

- Evaluation mode
- model.eval()
- torch.no_grad()
- Validation loss
- Validation metrics

19. Testing & Inference

- Loading trained models
- Inference
- Prediction
- Batch inference
- Single-sample inference

LEVEL 6 — MODEL MANAGEMENT

20. Saving & Loading

- state_dict
- torch.save
- torch.load
- Saving checkpoints
- Loading checkpoints

21. Checkpointing

- Model state
- Optimizer state
- Epoch
- Loss
- Best model checkpoint
- Resume training

22. Train / Eval Modes

- `model.train()`
- `model.eval()`
- Dropout behavior
- BatchNorm behavior

LEVEL 7 — DATA TRANSFORMATIONS

23. Torchvision

- torchvision
- Datasets
- Transforms
- Models
- Image utilities

24. Image Transforms

- Resize
- Crop
- Center crop
- Random crop
- Flip
- Rotation
- Color transformations
- Normalization
- Compose

25. Custom Transforms

- Writing custom transforms
- Transform pipelines
- Applying transformations conditionally

LEVEL 8 — CNNs WITH PYTORCH

26. Building CNNs

- Convolution layers
- Channels
- Kernel size
- Stride
- Padding
- Feature maps
- Pooling

27. CNN Training

- Image datasets
- CNN training loop
- Validation
- Classification
- Confusion matrix

28. Pretrained Models

- ResNet
- VGG
- EfficientNet
- MobileNet
- DenseNet

29. Transfer Learning

- Loading pretrained weights
- Freezing layers
- Replacing classifier
- Fine-tuning
- Differential learning rates

LEVEL 9 — ADVANCED TRAINING

30. Learning Rate Schedulers

- StepLR
- MultiStepLR
- ExponentialLR
- CosineAnnealing
- ReduceLROnPlateau
- OneCycleLR

31. Weight Initialization

- Xavier
- He initialization
- Custom initialization

32. Regularization

- Weight decay
- Dropout
- Early stopping
- Label smoothing

33. Mixed Precision

- Why mixed precision
- FP16
- BF16
- Automatic Mixed Precision
- autocast
- GradScaler

34. Gradient Techniques

- Gradient clipping
- Gradient accumulation
- Gradient checkpointing

LEVEL 10 — GPU & PERFORMANCE

35. *CUDA with PyTorch*

- CUDA devices
- GPU memory
- CPU vs GPU
- Device selection
- Multi-GPU basics
- CUDA synchronization

36. *Performance Optimization*

- Batch size optimization
- DataLoader optimization
- pin_memory
- num_workers
- GPU utilization
- Memory management

37. *Profiling*

- PyTorch Profiler
- CPU/GPU profiling
- Identifying bottlenecks
- Training performance analysis

LEVEL 11 — ADVANCED PYTORCH

38. *Custom Autograd*

- Custom backward functions
- torch.autograd.Function
- Forward/backward implementation

39. *Custom Layers*

- Custom nn.Module
- Learnable parameters
- Custom operations

40. *Hooks*

- Forward hooks
- Backward hooks
- Activation extraction
- Debugging models

41. *Advanced Tensor Operations*

- Broadcasting
- einsum
- Advanced indexing
- Tensor manipulation
- Memory layout

LEVEL 12 — MODERN PYTORCH

42. *torch.compile*

- Compilation
- Graph capture
- Compilation modes
- Performance benefits

43. *TorchScript*

- Scripting
- Tracing
- Model serialization

44. *Distributed Training*

- Data Parallelism
- DistributedDataParallel
- Multi-GPU training
- DistributedDataSampler

45. *FSDP*

- Fully Sharded Data Parallel
- Parameter sharding
- Memory efficiency
- Large-model training

LEVEL 13 — MODEL OPTIMIZATION & DEPLOYMENT

46. *Quantization*

- Quantization fundamentals
- Dynamic quantization
- Static quantization
- Quantization-aware training

47. *Pruning*

- Weight pruning
- Structured pruning
- Unstructured pruning

48. *Knowledge Distillation*

- Teacher model
- Student model
- Distillation loss
- Model compression

49. *ONNX*

- Exporting PyTorch models
- ONNX graph
- ONNX Runtime
- Inference

50. *Production Inference*

- Batch inference
- Real-time inference
- Model serving
- API inference
- Docker
- CPU/GPU inference

LEVEL 14 — PYTORCH ECOSYSTEM

51. *TorchVision*

- Datasets
- Transforms
- Pretrained models
- Detection models
- Segmentation models

52. *TorchMetrics*

- Metrics
- Classification metrics
- Detection metrics
- Segmentation metrics

53. *Experiment Tracking*

- TensorBoard
- MLflow
- Weights & Biases

54. *Reproducibility*

- Random seeds
- Deterministic operations
- Configuration management
- Environment management